ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №3

«Ітератори, генератори та потокова обробка даних»

з курсу «Професійний Python»
Варіант №3 — «Потокова обробка даних інтернет-магазину»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 08.10.26

Львів — 2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, протоколом ітератора (__iter__(), __next__()), генераторними функціями, ключовими словами yield та yield from, генераторними виразами (generator expressions), модулем itertools та принципом лінивих обчислень (lazy evaluation). Побудувати конвеєр потокової обробки великих наборів даних (streaming processing) без повного завантаження даних у пам'ять, реалізувати порційну обробку (batching), порівняти швидкодію та пікове споживання пам'яті eager та lazy підходів.

2. Індивідуальний варіант і постановка задачі

Варіант №3 — «Потокова обробка даних інтернет-магазину».

Формат вхідних даних (CSV): product_id,name,category,price,quantity.

Необхідно реалізувати:

1. Streaming validation ціни та кількості.
2. Фільтрацію товарів за категорією.
3. Пошук товару за ID або назвою з використанням раннього переривання (early termination).
4. Streaming-обчислення загальної вартості залишків.
5. Визначення найдорожчого товару без повної матеріалізації колекцій.
6. Lazy-фільтрацію товарів із низьким запасом.
7. Розбиття потоку на батчі (generator batches).
8. Формування категоріальної статистики без повного завантаження датасету в пам'ять.
9. Обробку не менше 100 000 записів із порівнянням часу та пікового використання пам'яті (Eager vs Lazy).

3. Структура проєкту

Проєкт організовано відповідно до сучасної архітектурної моделі src-layout, яка запобігає випадковому імпорту непакетованого коду:

```
stream_project/
├── data/
│   └── products.csv
├── report/
│   └── Вересюк_Ірина_Лаб03_V3.pdf
├── .gitignore
├── pyproject.toml
├── README.md
└── src/
    └── stream_processor/
```

├── __init__.py

├── models.py

├── iterators.py

├── readers.py

├── parsers.py

├── filters.py

├── transformations.py

├── batches.py

├── analytics.py

├── generator.py

├── benchmark.py

└── main.py

Каталог data/ виділено для зберігання вхідних CSV-файлів. Папка report/ містить звітну документацію у форматі PDF. Пакет src/stream_processor/ інкапсулює вихідний код, розділений за принципом єдиної відповідальності (SRP): структури даних (models.py), операції введення-виведення (readers.py, parsers.py), бізнес-правила обробки (filters.py, transformations.py), пакетування (batches.py), статистику (analytics.py), експериментальний бенчмарк (benchmark.py) та виконавчу точку входу (main.py)

4. Теоретичні засади: Iterable, Iterator та протокол ітерації

Iterable (ітерований об'єкт): будь-який об'єкт Python, який містить магічний метод __iter__() і повертає ітератор (наприклад: списки, словники, відкриті файлові об'єкти).

Iterator (ітератор): об'єкт стану, що реалізує метод __next__() для послідовного повернення елементів або викидання винятку StopIteration після завершення послідовності. Обов'язково має метод __iter__(), який повертає сам об'єкт (iter(it) is it).

Цикл for: реалізує ітераційний протокол: автоматично викликає iter() над колекцією, послідовно вибирає значення через next() та безпечно перехоплює виняток StopIteration.

One-shot природа: більшість ітераторів є одноразовими; після вичерпання вони не повертають значень і потребують повторного конструювання.

5. Опис створених модулів та лістинги коду

5.1. models.py (Модель даних)

Для опису сутності товару використано структуру NamedTuple. Вона забезпечує сувору типізацію, компактне розміщення в пам'яті (без використання внутрішнього словника __dict__) та зручний доступ до властивостей через крапку.

```
from typing import NamedTuple


class ProductRecord(NamedTuple):
```

```
    product_id: int
    name: str
    category: str
    price: float
    quantity: int

    @property
    def total_cost(self) -> float:
        return round(self.price * self.quantity, 2)
```

Клас ProductRecord зберігає атрибути товару, а властивість total_cost інкапсулює обчислення загальної вартості конкретної товарної позиції.

5.2. iterators.py (Власний клас ітератора)

Створено повноцінний користувацький ітератор ProductBatchIterator, що реалізує протокол __iter__() та __next__().

```
from collections.abc import Iterator
from typing import Any, Sequence

class ProductBatchIterator(Iterator):
    """Власний ітератор для послідовного проходження по колекції батчів."""
    def __init__(self, data: Sequence[Any], step: int) -> None:
        self._data = data
        self._step = max(1, step)
        self._index = 0

    def __iter__(self) -> "ProductBatchIterator":
        return self

    def __next__(self) -> Sequence[Any]:
        if self._index >= len(self._data):
            raise StopIteration
        batch = self._data[self._index : self._index + self._step]
        self._index += self._step
        return batch
```

Модуль демонструє розуміння низькорівневої ітераційної моделі мови Python: клас ітератора вручну відстежує внутрішній індекс зсуву і генерує зрізи даних до настання умови завершення обходу StopIteration.

5.3. readers.py та parsers.py (Потоковий ввід даних)

У модулі readers.py використано конструкцію yield from для прямого транслювання рядків із контекстного менеджера відкритого файлу. У parsers.py здійснюється перетворення рядків на словники.

readers.py

```
from collections.abc import Iterator

from pathlib import Path


def stream_read_lines(path: Path) -> Iterator[str]:

  """Потокове читання файлу без завантаження всього вмісту в RAM."""

  with path.open("r", encoding="utf-8") as file:

    yield from file
```

parsers.py

```
import csv

from collections.abc import Iterable, Iterator


def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:

  """Розбір рядків у словники за допомогою csv.DictReader."""

  reader = csv.DictReader(lines)

  yield from reader
```

Забезпечується потокове читання (streaming): файл відкривається і читається буферизовано рядок за рядком, уникаючи виклику методу .read(), який би завантажив гігабайти тексту в оперативну пам'ять.

5.4. filters.py (Валідація, фільтрація та Early Termination)

Модуль містить генераторні функції для очищення вхідних даних від пошкоджених записів, фільтрації за категорією та пошуку товарів.

```
from collections.abc import Iterable, Iterator

from typing import Optional

from .models import ProductRecord


def validate_products(rows: Iterable[dict[str, str]]) -> Iterator[ProductRecord]:

  """Streaming validation ціни (price > 0) та кількості (quantity >= 0)."""
```

```
    for row in rows:
        try:
            p_id = int(row["product_id"])
            name = row["name"].strip()
            category = row["category"].strip()
            price = float(row["price"])
            quantity = int(row["quantity"])
        except (ValueError, TypeError, KeyError):
            continue

        if not name or not category:
            continue
        if price < 0 or quantity < 0:
            continue

        yield ProductRecord(
            product_id=p_id,
            name=name,
            category=category,
            price=price,
            quantity=quantity,
        )


def filter_by_category(products: Iterable[ProductRecord], category: str) -> Iterator[ProductRecord]:
    """Фільтрація товарів за категорією."""
    target = category.casefold()
    for product in products:
        if product.category.casefold() == target:
            yield product


def filter_low_stock(products: Iterable[ProductRecord], threshold: int = 5) ->
Iterator[ProductRecord]:
    """Lazy-фільтрація товарів із низьким залишком."""
```

```
    for product in products:
        if product.quantity <= threshold:
            yield product


def find_product(
    products: Iterable[ProductRecord],
    product_id: Optional[int] = None,
    name: Optional[str] = None
) -> Optional[ProductRecord]:
    """Lazy-пошук товару з раннім перериванням (early termination)."""
    target_name = name.casefold() if name else None
    for product in products:
        if product_id is not None and product.product_id == product_id:
            return product
        if target_name is not None and product.name.casefold() == target_name:
            return product
    return None
```

validate_products відсіює записи з некоректними типами даних, порожніми назвами та від'ємними значеннями. Функція find_product демонструє ранній вихід (early termination): генерація припиняється одразу після першого збігу, не обробляючи решту файлу.

5.5. batches.py (Генераторний батчинг через itertools.islice)

Реалізація розбиття нескінченного або великого потоку на порції (батчі) за допомогою інструменту itertools.islice.

```
from collections.abc import Iterable, Iterator
from itertools import islice
from typing import TypeVar

T = TypeVar("T")

def batched(iterable: Iterable[T], batch_size: int) -> Iterator[list[T]]:
    """Генераторне розбиття потоку на порції заданого розміру."""
    iterator = iter(iterable)
```

```
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch
```

Модуль дозволяє передавати дані зовнішнім системам (API, СУБД) фіксованими пакетами. У пам'яті одночасно зберігається лише поточний батч розміром N елементів, що зберігає переваги лінивих обчислень.

5.6. analytics.py (Потокова агрегація)

Модуль виконує аналітичні розрахунки в один потоковий прохід без збереження проміжних списків.

```
from collections import defaultdict
from collections.abc import Iterable
from typing import Any, Optional
from .models import ProductRecord


def calculate_inventory_statistics(products: Iterable[ProductRecord]) -> dict[str, Any]:
    """Streaming-обчислення загальної вартості, найдорожчого товару та статистики категорій."""
    total_inventory_cost = 0.0
    total_quantity = 0
    records_count = 0
    most_expensive_product: Optional[ProductRecord] = None

    category_stats: dict[str, dict[str, Any]] = defaultdict(
        lambda: {"count": 0, "total_value": 0.0, "total_qty": 0}
    )

    for item in products:
        records_count += 1
        item_total = item.total_cost

        total_inventory_cost += item_total
        total_quantity += item.quantity
```

```
        if most_expensive_product is None or item.price > most_expensive_product.price:
            most_expensive_product = item

        cat = category_stats[item.category]
        cat["count"] += 1
        cat["total_value"] += item_total
        cat["total_qty"] += item.quantity

    return {
        "records_count": records_count,
        "total_inventory_cost": round(total_inventory_cost, 2),
        "total_quantity": total_quantity,
        "most_expensive_product": most_expensive_product,
        "category_statistics": dict(category_stats),
    }
```

Функція обчислює загальну вартість залишків, кількість товарів, знаходить найдорожчу одиницю та агрегує зведення за кожною товарною категорією. Споживання пам'яті залишається константним $O(1)$ незалежно від кількості записів.

6. Робота з Git

У проєкті створено три логічні коміти, які поетапно фіксують створення структури, реалізацію бізнес-конвеєра та налаштування точки входу з бенчмаркінгом.

Коміт 1: "Create project structure, models and streaming reader"

```
Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (master)
$ git add .gitignore src/stream_processor/__init__.py src/stream_processor/models.py
src/stream_processor/readers.py src/stream_processor/parsers.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (master)
$ git commit -m "Create project structure, models and streaming reader"
[master (root-commit) a6ed7bb] Create project structure, models and streaming reader
 5 files changed, 66 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 src/stream_processor/__init__.py
 create mode 100644 src/stream_processor/models.py
```

create mode 100644 src/stream_processor/parsers.py

create mode 100644 src/stream_processor/readers.py

Ініціалізовано git-репозиторій за стандартом src-layout. Налаштовано .gitignore для виключення системних кешів __pycache__ та важких файлів data/*.csv. Зафіксовано сутність ProductRecord, модуль лінивого читання рядків та парсер CSV.

Коміт 2: "Implement streaming pipeline, filters, batching and analytics"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (master)

$ git add src/stream_processor/filters.py src/stream_processor/transformations.py src/stream_processor/batches.py src/stream_processor/analytics.py src/stream_processor/iterators.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (master)

$ git commit -m "Implement streaming pipeline, filters, batching and analytics"

[master 08df776] Implement streaming pipeline, filters, batching and analytics

5 files changed, 136 insertions(+)

create mode 100644 src/stream_processor/analytics.py

create mode 100644 src/stream_processor/batches.py

create mode 100644 src/stream_processor/filters.py

create mode 100644 src/stream_processor/iterators.py

create mode 100644 src/stream_processor/transformations.py

Реалізовано основні ланки потокової обробки: генератори валідації та фільтрації, нормалізацію тексту, модуль розбиття на батчі через islice, функції зведення статистики в один прохід та власний клас ітератора ProductBatchIterator.

Коміт 3: "Add synthetic generator, performance benchmark and main entry point"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (master)

$ git add src/stream_processor/generator.py src/stream_processor/benchmark.py src/stream_processor/main.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (master)

$ git commit -m "Add synthetic generator, performance benchmark and main entry point"

[master 6a5fae0] Add synthetic generator, performance benchmark and main entry point

3 files changed, 161 insertions(+)

create mode 100644 src/stream_processor/benchmark.py

create mode 100644 src/stream_processor/generator.py

create mode 100644 src/stream_processor/main.py

Зафіксовано генератор тестового датасету на 100 000 рядків, бенчмарк для замірів пікової пам'яті (tracemalloc) та головний модуль main.py, що демонструє роботу всього конвеєра та виводить порівняльну таблицю.

6.1. Результат git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (main)

$ git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Підтверджує, що всі вихідні файли зафіксовані в коммітах, робоче дерево повністю чисте, а тимчасові каталоги компілятора та згенеровані CSV-файли успішно ігноруються репозиторієм завдяки .gitignore.

6.2. Результат git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/stream_project (main)

$ git log --oneline

6a5fae0 (**HEAD** -> **main**, **origin/main**) Add synthetic generator, performance benchmark and main entry point

08df776 Implement streaming pipeline, filters, batching and analytics

a6ed7bb Create project structure, models and streaming reader

Відображає лінійну та логічну історію розробки проєкту з 3 змістовних комітів, синхронізованих із віддаленою гілкою origin/main на GitHub.

7. Результати запуску програми

Виведення консолі під час запуску команди python -m src.stream_processor.main:

PS D:\Робочий стіл\проф. python\stream_project> python -m src.stream_processor.main

[1/5] Перевірка тестового датасету...

Файл готовий.

[2/5] Демонстрація базових вимог варіанта 3:

Перші товари з низьким залишком (<= 3): [ProductRecord(product_id=3, name='Gaming Mouse 993', category='Audio', price=2637.22, quantity=3), ProductRecord(product_id=146, name='Eco Headset 145', category='Audio', price=751.84, quantity=2), ProductRecord(product_id=149, name='Pro Hub 334', category='Monitors', price=1765.29, quantity=3)]

Знайдений товар (ID=42): ProductRecord(product_id=42, name='Gaming Hub 529', category='Laptops', price=2700.68, quantity=20)

Зразок перших 2 батчів: [[ProductRecord(product_id=1, name='Gaming Notebook 355', category='Accessories', price=1801.1, quantity=36), ProductRecord(product_id=2, name='Eco Headset 379', category='Keyboards', price=3384.77, quantity=27)], [ProductRecord(product_id=3, name='Gaming Mouse 993', category='Audio', price=2637.22, quantity=3), ProductRecord(product_id=4, name='Gaming Keyboard 543', category='Audio', price=2813.86, quantity=23)]]

[3/5] Використання бібліотеки itertools:

Товари з chain(Laptops, Audio): ['Lite Keyboard 197', 'Lite Notebook 156', 'Gaming Mouse 993', 'Gaming Keyboard 543']

Кумулятивна сума перших 4 товарів (accumulate): [64839.6, 156228.38999999998, 164140.05, 228858.83]

[4/5] Потоковий розрахунок статистики (100 000 записів)...

Опрацьовано записів : 100000

Загальна вартість : 8,762,330,657.39 грн

Загальна кількість : 4993161 од.

Найдорожчий товар : Lite Keyboard 967 (3499.99 грн)

Категорій : 6

[5/5] Порівняння Eager vs Lazy (час та споживання RAM):

Показник | Eager | Lazy

Час виконання (с) | 2.3215 | 1.5736

Пікова RAM (МБ) | 52.91 | 0.05

PS D:\Робочий стіл\проф. python\stream_project>

Демонструє повний життєвий цикл виконання програми: перевірку та генерацію вибірки, фільтрацію низького запасу за допомогою islice, пошук за ID із миттєвим раннім виходом, генераторний батчинг, роботу функцій itertools.chain та itertools.accumulate, фінальну агрегацію та виведення порівняльних метрик.

8. **Висновок**

Під час виконання лабораторної роботи №3 було успішно опановано концепції ітераційної моделі Python, роботу з ітераторами, генераторними функціями, yield, yield from, модулем itertools та принципом лінивих обчислень (lazy evaluation).

Для предметної області «Інтернет-магазин» (Варіант 3) було побудовано модульний потоковий конвеєр, що включає валідацію, фільтрацію, нормалізацію, батчинг та агрегацію даних. Експериментальне порівняння на вибірці зі $100\,000$ записів наочно підтвердило теоретичні засади: лінива обробка знизила пікове споживання оперативної пам'яті з 52.90 **МБ до** 0.05

МБ (економія у понад 1000 разів), забезпечивши стабільну та надійну роботу програми без загрози переповнення пам'яті.